



Day 01 — The Logic of Programming and the R Environment

Setup, package management, calculations, vectors,
matrices, lists, and piping

AUTHORS

Johann-Friedrich Salzmänn  

Joshua Elias Schmidt Rodrigues  

DATE

2026-08-31

Topics:

Where we left off	2
Variable types	5
Vectors	7
Piping	9
Sequences	11
Indexing & subsetting	11
A first look at if/else	13
Matrices	14
Lists	15
A first data frame	17
Try it: populist support across federal states	21

Where we left off

Same commands as the editor, now click-to-run — run them again here.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

Loading required package: pacman

```
p_load(tidyverse)
```

```
# normal math
```

```
1 + 1
```

> OUTPUT

```
[1] 2
```

```
5 * 2
```

> OUTPUT

```
[1] 10
```

```
5 / 2
```

> OUTPUT

```
[1] 2.5
```

```
5^2
```

> OUTPUT

```
[1] 25
```

```
(1 + 2) + 2^2 * 3
```

> OUTPUT

```
[1] 15
```

```
0.5
```

```
>_ OUTPUT
```

```
[1] 0.5
```

```
.5
```

```
>_ OUTPUT
```

```
[1] 0.5
```

```
# functions for calculations, help function  
sqrt(4)
```

```
>_ OUTPUT
```

```
[1] 2
```

```
?sqrt
```

```
round(3.142, digits = 1)
```

```
>_ OUTPUT
```

```
[1] 3.1
```

```
round(3.142, 1)
```

```
>_ OUTPUT
```

```
[1] 3.1
```

```
# assignments: both work equally  
x <- 5  
y = 5  
  
income = 1500  
job = "Political Scientist"
```

```
# print() keeps the quotes, cat() doesn't  
print(job)
```

```
>_ OUTPUT
```

```
[1] "Political Scientist"
```

```
cat(job)
```

>_ OUTPUT

```
Political Scientist
```

```
# paste() glues pieces together, space-separated by default  
paste("I am a", job, "and I earn", income, "per month.")
```

>_ OUTPUT

```
[1] "I am a Political Scientist and I earn 1500 per month."
```

```
paste0("I am a ", job, " and I earn ", income, "€ per month.") # paste0() = paste(sep  
= "")
```

>_ OUTPUT

```
[1] "I am a Political Scientist and I earn 1500€ per month."
```

```
paste("Some Info", job, income, sep = " - ")
```

>_ OUTPUT

```
[1] "Some Info - Political Scientist - 1500"
```

```
# str_glue() interpolates variables straight into {}  
str_glue("I am a {job} and I earn {income}€ per month.")
```

>_ OUTPUT

```
I am a Political Scientist and I earn 1500€ per month.
```

```
str_glue("Some Info - {job} - {income}")
```

>_ OUTPUT

```
Some Info - Political Scientist - 1500
```

```
str_glue(
  "I am a {job_title} and I earn {monthly_income} € per month.",
  job_title = job, # custom {placeholder}, not tied to an existing variable
  monthly_income = income + 200
)
```

>_ OUTPUT

I am a Political Scientist and I earn 1700 € per month.

From here on, <- is our default assignment operator.

```
voted <- 24000
eligible <- 32101

turnout <- voted / eligible * 100
turnout
```

>_ OUTPUT

[1] 74.76403

```
round(turnout, digits = 1)
```

>_ OUTPUT

[1] 74.8

```
turnout
```

>_ OUTPUT

[1] 74.76403

```
rounded <- round(turnout, digits = 1)
rounded
```

>_ OUTPUT

[1] 74.8

Variable types

Every value has a type — R guesses it for you, but you should know it.

```
vote_share <- 42.3  
class(vote_share)
```

>_ OUTPUT

```
[1] "numeric"
```

```
party <- "Greens"  
class(party)
```

>_ OUTPUT

```
[1] "character"
```

```
won_election <- TRUE  
class(won_election)
```

>_ OUTPUT

```
[1] "logical"
```

```
region <- factor("East")  
class(region)
```

>_ OUTPUT

```
[1] "factor"
```

Convert between types where it makes sense.

```
vote_share_string <- as.character(vote_share)  
class(vote_share_string)
```

>_ OUTPUT

```
[1] "character"
```

```
as.numeric("42.3")
```

>_ OUTPUT

```
[1] 42.3
```

```
as.numeric("Greens") # not every conversion works
```

⚠ WARNING

Warning: NAs introduced by coercion

>_ OUTPUT

```
[1] NA
```

Numeric (double) vs. integer.

```
16L
```

>_ OUTPUT

```
[1] 16
```

```
as.integer(16)
```

>_ OUTPUT

```
[1] 16
```

```
class(16L)
```

>_ OUTPUT

```
[1] "integer"
```

```
class(16)
```

>_ OUTPUT

```
[1] "numeric"
```

Vectors

One object, many values of the same type.

```
parties <- c("SPD", "CDU", "Greens", "FDP", "AfD")  
vote_share <- c(25.7, 24.1, 14.7, 11.5, 10.3)  
  
length(parties)
```

>_ OUTPUT

```
[1] 5
```

```
length(vote_share)
```

```
>_ OUTPUT
```

```
[1] 5
```

```
mean(vote_share)
```

```
>_ OUTPUT
```

```
[1] 17.26
```

```
min(vote_share)
```

```
>_ OUTPUT
```

```
[1] 10.3
```

```
max(vote_share)
```

```
>_ OUTPUT
```

```
[1] 25.7
```

```
sum(vote_share)
```

```
>_ OUTPUT
```

```
[1] 86.3
```

Vectorised math — applies element by element.

```
vote_share / 100 # convert into actual shares (%)
```

```
>_ OUTPUT
```

```
[1] 0.257 0.241 0.147 0.115 0.103
```

Named vectors attach a label to each value.


```

vote_share_named <- c(
  SPD = 25.7,
  CDU = 24.1,
  Greens = 14.7,
  FDP = 11.5,
  AfD = 10.3
)
vote_share_named

```

>_ OUTPUT

SPD	CDU	Greens	FDP	AfD
25.7	24.1	14.7	11.5	10.3

```

vote_share_named <- setNames(vote_share, parties)
vote_share_named

```

>_ OUTPUT

SPD	CDU	Greens	FDP	AfD
25.7	24.1	14.7	11.5	10.3

Mixing types coerces everything to the most flexible type.

```
c(1, 2, "three")
```

>_ OUTPUT

```
[1] "1"    "2"    "three"
```

Boolean shortcuts.

```
c(TRUE, T, FALSE, F)
```

>_ OUTPUT

```
[1] TRUE TRUE FALSE FALSE
```

Piping

Same calls, written left to right instead of inside out.

```
mean(vote_share)
```

>_ OUTPUT

```
[1] 17.26
```

```
vote_share |> mean()
```

>_ OUTPUT

```
[1] 17.26
```

```
mean(vote_share, na.rm = TRUE)
```

>_ OUTPUT

```
[1] 17.26
```

```
vote_share |> mean(na.rm = TRUE)
```

>_ OUTPUT

```
[1] 17.26
```

```
round(mean(vote_share), digits = 1)
```

>_ OUTPUT

```
[1] 17.3
```

```
vote_share |> mean() |> round(digits = 1)
```

>_ OUTPUT

```
[1] 17.3
```

Magrittr pipe — same idea, older syntax you'll still see around.

```
p_load(magrittr)
vote_share %>% mean() %>% round(1)
```

>_ OUTPUT

```
[1] 17.3
```

%<>% pipes into and reassigns the same object.

```
vote_share
```

>_ OUTPUT

```
[1] 25.7 24.1 14.7 11.5 10.3
```

```
vote_share %<>% round(0)
vote_share
```

>_ OUTPUT

```
[1] 26 24 15 12 10
```

Sequences

Quick ways to build a run of numbers.

```
1:10
```

>_ OUTPUT

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
seq(from = 0, to = 100, by = 10)
```

>_ OUTPUT

```
[1] 0 10 20 30 40 50 60 70 80 90 100
```

```
weeks <- 1:12
```

Indexing & subsetting

Pulling specific elements out of a vector.

Base R

```
parties[1] # first element
```

>_ OUTPUT

```
[1] "SPD"
```

```
parties[length(parties)] # last element
```

>_ OUTPUT

```
[1] "AfD"
```

```
parties[-1] # everything except the first
```

> OUTPUT

```
[1] "CDU"      "Greens" "FDP"      "AfD"
```

```
parties[1:3] # first three elements
```

> OUTPUT

```
[1] "SPD"      "CDU"      "Greens"
```

```
vote_share_named[["AfD"]] # named element
```

> OUTPUT

```
[1] 10.3
```

Tidyverse

```
first(parties) # first element
```

> OUTPUT

```
[1] "SPD"
```

```
last(parties) # last element
```

> OUTPUT

```
[1] "AfD"
```

```
tail(parties, -1) # everything except the first
```

> OUTPUT

```
[1] "CDU"      "Greens" "FDP"      "AfD"
```

```
head(parties, 3) # first three elements
```

> OUTPUT

```
[1] "SPD"      "CDU"      "Greens"
```

```
vote_share_named |> pluck("AfD") # named element
```

>_ OUTPUT

```
[1] 10.3
```

```
# comparisons return logical vectors  
vote_share > 15
```

>_ OUTPUT

```
[1] TRUE TRUE FALSE FALSE FALSE
```

```
which(vote_share > 15)
```

>_ OUTPUT

```
[1] 1 2
```

Base R

```
parties[vote_share > 15] # subsetting with a condition
```

>_ OUTPUT

```
[1] "SPD" "CDU"
```

Tidyverse

```
parties |> keep(vote_share > 15) # subsetting with a condition
```

>_ OUTPUT

```
[1] "SPD" "CDU"
```

A first look at if/else

More on this later this week.

```
threshold <- 5

x <- 3

if (x > threshold) {
  cat("Is greater than the threshold")
} else {
  cat("Is not greater than the threshold")
}
```

> OUTPUT

Is not greater than the threshold

```
if (first(vote_share) > threshold) {
  # If condition is true, execute:
  str_glue(
    "{first_party} enters parliament",
    first_party = first(parties)
  ) |> cat()
} else {
  # If condition is false, execute:
  cat(str_glue("{first_party} misses the threshold", first_party = first(parties)))
}
```

> OUTPUT

SPD enters parliament

Matrices

Bundling equal-length vectors together, by row or by column. Rarely required nowadays unless doing matrix algebra.

```
party_id <- c(1, 2, 3)
seats_2021 <- c(206, 152, 118)
seats_2025 <- c(120, 164, 85)

seats_matrix <- cbind(party_id, seats_2021, seats_2025)
seats_matrix
```

> OUTPUT

	party_id	seats_2021	seats_2025
[1,]	1	206	120
[2,]	2	152	164
[3,]	3	118	85

```
dim(seats_matrix)
```

```
>_ OUTPUT
```

```
[1] 3 3
```

```
nrow(seats_matrix)
```

```
>_ OUTPUT
```

```
[1] 3
```

```
ncol(seats_matrix)
```

```
>_ OUTPUT
```

```
[1] 3
```

```
rowSums(seats_matrix)
```

```
>_ OUTPUT
```

```
[1] 327 318 206
```

```
colSums(seats_matrix)
```

```
>_ OUTPUT
```

```
party_id seats_2021 seats_2025  
      6         476         369
```

Lists

Unlike vectors, list elements can be different types and lengths.

```
merkel <- list(  
  name = "Angela Merkel",  
  party = "CDU",  
  years_in_office = 16,  
  chancellor = TRUE  
)
```

Base R

```
merkel$party
```

>_ OUTPUT

```
[1] "CDU"
```

```
merkel[["years_in_office"]]
```

>_ OUTPUT

```
[1] 16
```

Tidyverse

```
merkel |> pluck("name")
```

>_ OUTPUT

```
[1] "Angela Merkel"
```

```
merkel %$% chancellor
```

>_ OUTPUT

```
[1] TRUE
```

Lists can nest — a list of lists. Have a look at your IDE's variable inspector.

```
government <- list(  
  chancellor = merkel,  
  coalition_parties = c("CDU", "SPD")  
)
```

Base R

```
government$chancellor$party
```

>_ OUTPUT

```
[1] "CDU"
```

```
government$coalition_parties
```

>_ OUTPUT

```
[1] "CDU" "SPD"
```


Tidyverse

```
government |> pluck("chancellor", "party")
```

>_ OUTPUT

```
[1] "CDU"
```

```
government |> pluck("coalition_parties")
```

>_ OUTPUT

```
[1] "CDU" "SPD"
```

A first data frame

A table: columns are vectors, rows are observations. Full tidyverse treatment (tibbles, dplyr) comes Day 3 — for now, a first tibble.

```
parties_df <- tibble(  
  party = parties,  
  vote_share = vote_share,  
  in_government = c(TRUE, FALSE, TRUE, FALSE, FALSE)  
)  
parties_df
```

>_ OUTPUT

```
# A tibble: 5 × 3  
  party   vote_share in_government  
  <chr>      <dbl>   <lgl>  
1 SPD         26    TRUE  
2 CDU         24   FALSE  
3 Greens      15    TRUE  
4 FDP         12   FALSE  
5 AfD         10   FALSE
```

Base R

```
parties_df$vote_share
```

>_ OUTPUT

```
[1] 26 24 15 12 10
```

```
parties_df[parties_df$in_government, ]
```

> OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 SPD      26 TRUE
2 Greens   15 TRUE
```

```
parties_df[parties_df$in_government == TRUE, ]
```

> OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 SPD      26 TRUE
2 Greens   15 TRUE
```

```
parties_df[1, ]
```

> OUTPUT

```
# A tibble: 1 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 SPD      26 TRUE
```

```
parties_df[nrow(parties_df), ]
```

> OUTPUT

```
# A tibble: 1 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 AfD      10 FALSE
```

```
parties_df[1:2, ]
```

> OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 SPD      26 TRUE
2 CDU      24 FALSE
```

```
parties_df[(nrow(parties_df) - 1):nrow(parties_df), ]
```

> OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 FDP      12 FALSE
2 AfD      10 FALSE
```

```
parties_df[c(2, 3), ]
```

> OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 CDU      24 FALSE
2 Greens   15  TRUE
```

Tidyverse

```
parties_df |> pull(vote_share)
```

> OUTPUT

```
[1] 26 24 15 12 10
```

```
parties_df |> filter(in_government)
```

> OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 SPD      26  TRUE
2 Greens   15  TRUE
```

```
parties_df |> filter(in_government == TRUE)
```

> OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
<chr>   <dbl> <lgl>
1 SPD      26  TRUE
2 Greens   15  TRUE
```

```
parties_df |> first()
```

>_ OUTPUT

```
# A tibble: 1 × 3
  party vote_share in_government
  <chr>      <dbl> <lgl>
1 SPD          26 TRUE
```

```
parties_df |> last()
```

>_ OUTPUT

```
# A tibble: 1 × 3
  party vote_share in_government
  <chr>      <dbl> <lgl>
1 AfD          10 FALSE
```

```
parties_df |> slice_head(n = 2)
```

>_ OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
  <chr>      <dbl> <lgl>
1 SPD          26 TRUE
2 CDU          24 FALSE
```

```
parties_df |> slice_tail(n = 2)
```

>_ OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
  <chr>      <dbl> <lgl>
1 FDP          12 FALSE
2 AfD          10 FALSE
```

```
parties_df |> slice(c(2, 3))
```

>_ OUTPUT

```
# A tibble: 2 × 3
  party vote_share in_government
  <chr>      <dbl> <lgl>
1 CDU          24 FALSE
2 Greens       15 TRUE
```

Try it: populist support across federal states

Same shape as `parties_df`, plus a `state` column — fictional AfD/BSW results per Bundesland (federal state); both count as “populist” here.

Your task: filter the table below down to just the AfD/BSW rows, then find out — does any state cross an absolute majority ($\geq 50\%$)? Which states cross the blocking minority (Sperrminorität, $\geq 33.3\%$)? And what’s the average vote share among rows that clear the 5% entry threshold?

```

polls_by_state <- tribble(
  ~state, ~party, ~vote_share,
  "Sachsen", "AfD", 32.4,
  "Sachsen", "BSW", 11.8,
  "Sachsen", "CDU", 31.9,
  "Sachsen", "SPD", 7.5,
  "Thüringen", "AfD", 34.6,
  "Thüringen", "BSW", 13.1,
  "Thüringen", "CDU", 23.6,
  "Thüringen", "SPD", 6.1,
  "Brandenburg", "AfD", 29.2,
  "Brandenburg", "BSW", 12.4,
  "Brandenburg", "CDU", 15.8,
  "Brandenburg", "SPD", 30.9,
  "Bayern", "AfD", 18.0,
  "Bayern", "BSW", 6.5,
  "Bayern", "CDU", 37.0,
  "Bayern", "SPD", 8.4,
  "Baden-Württemberg", "AfD", 15.4,
  "Baden-Württemberg", "BSW", 4.2,
  "Baden-Württemberg", "CDU", 24.1,
  "Baden-Württemberg", "SPD", 9.7,
  "Nordrhein-Westfalen", "AfD", 14.2,
  "Nordrhein-Westfalen", "BSW", 5.1,
  "Nordrhein-Westfalen", "CDU", 34.6,
  "Nordrhein-Westfalen", "SPD", 22.1,
  "Bremen", "AfD", 9.8,
  "Bremen", "BSW", 3.2,
  "Bremen", "CDU", 22.5,
  "Bremen", "SPD", 26.0,
  "Hamburg", "AfD", 8.9,
  "Hamburg", "BSW", 2.8,
  "Hamburg", "CDU", 22.9,
  "Hamburg", "SPD", 30.2
)
polls_by_state

```

> OUTPUT

```
# A tibble: 32 × 3
  state      party vote_share
  <chr>      <chr>     <dbl>
1 Sachsen   AfD         32.4
2 Sachsen   BSW         11.8
3 Sachsen   CDU         31.9
4 Sachsen   SPD          7.5
5 Thüringen AfD         34.6
6 Thüringen BSW         13.1
7 Thüringen CDU         23.6
8 Thüringen SPD          6.1
9 Brandenburg AfD        29.2
10 Brandenburg BSW        12.4
# i 22 more rows
```

```
populist_results <- polls_by_state |> filter(party %in% c("AfD", "BSW"))
populist_results
```

> OUTPUT

```
# A tibble: 16 × 3
  state      party vote_share
  <chr>      <chr>     <dbl>
1 Sachsen   AfD         32.4
2 Sachsen   BSW         11.8
3 Thüringen AfD         34.6
4 Thüringen BSW         13.1
5 Brandenburg AfD        29.2
6 Brandenburg BSW        12.4
7 Bayern    AfD          18
8 Bayern    BSW          6.5
9 Baden-Württemberg AfD        15.4
10 Baden-Württemberg BSW          4.2
11 Nordrhein-Westfalen AfD        14.2
12 Nordrhein-Westfalen BSW          5.1
13 Bremen    AfD          9.8
14 Bremen    BSW          3.2
15 Hamburg    AfD          8.9
16 Hamburg    BSW          2.8
```

An absolute majority — none reach it here.

```
majority <- populist_results |> filter(vote_share >= 50)
majority
```

> OUTPUT

```
# A tibble: 0 × 3
# i 3 variables: state <chr>, party <chr>, vote_share <dbl>
```

```
nrow(majority)
```

>_ OUTPUT

```
[1] 0
```

Over the blocking minority (Sperrminorität): can block a 2/3-majority vote.

```
blocking_minority <- populist_results |> filter(vote_share >= 33.3)
blocking_minority
```

>_ OUTPUT

```
# A tibble: 1 × 3
  state      party vote_share
<chr>    <chr>    <dbl>
1 Thüringen AfD          34.6
```

```
blocking_minority |> pull(state)
```

>_ OUTPUT

```
[1] "Thüringen"
```

Over the 5% entry threshold.

```
cleared_threshold <- populist_results |> filter(vote_share >= 5)
cleared_threshold |> pull(state) |> unique()
```

>_ OUTPUT

```
[1] "Sachsen"          "Thüringen"        "Brandenburg"
[4] "Bayern"           "Baden-Württemberg" "Nordrhein-Westfalen"
[7] "Bremen"           "Hamburg"
```

```
cleared_threshold |> pull(vote_share) |> mean()
```

>_ OUTPUT

```
[1] 16.26154
```